Retrieval-Augmented Generation: A Short Overview

1. What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines a retrieval system with a large language model. Instead of asking the model to answer from the knowledge stored in its weights, the system first searches an external collection of documents for passages related to the question and then gives those passages to the model as context.

The main benefit is grounding. Because the answer is built from retrieved text, it can be traced back to a source page, and the model is less likely to invent facts. A second benefit is freshness: updating the knowledge only requires re-indexing documents, not retraining the model.

2. The pipeline

A basic RAG pipeline has five stages: loading, chunking, embedding, retrieval and generation. Loading extracts text from files such as PDFs. Chunking splits that text into small overlapping pieces. Embedding converts every chunk into a vector. Retrieval finds the vectors closest to the question vector. Generation sends the question and the retrieved chunks to a language model, which writes the final answer.

Each stage has settings that change the quality of the result, and most of the effort in a real project goes into tuning them rather than into the model itself.

3. Chunking

Chunk size is a trade-off. Small chunks of a few hundred characters give precise matches but may lose the surrounding context. Large chunks keep more context but dilute the meaning of the vector, so unrelated content can match. A common starting point is 500 to 1000 characters with an overlap of 10 to 20 percent, so that a sentence cut at a boundary still appears whole in the neighbouring chunk.

Splitting on sentence or paragraph boundaries usually gives cleaner chunks than splitting at a fixed number of characters. Each chunk should also store metadata such as the file name and page number, so the final answer can cite its source.

4. Embeddings

An embedding model maps text to a point in a high-dimensional space so that texts with similar meaning end up close together. The sentence-transformers model all-MiniLM-L12-v2 produces vectors with 384 dimensions and truncates its input at 128 tokens by default, which is why chunks for this model are kept short.

The same model must be used for the documents and for the question. Vectors from two different models live in different spaces and cannot be compared. When vectors are normalized to unit length, the dot product equals cosine similarity, which is the usual measure of closeness.

5. Vector search

Exact search compares the question vector with every stored vector. This is fine for a few thousand chunks but becomes slow for millions. Approximate nearest neighbour (ANN) methods trade a small loss in accuracy for a large gain in speed.

ChromaDB uses an HNSW graph index. FAISS offers several index types. IndexIVFFlat, the inverted file index, first clusters all vectors into nlist groups using k-means. At query time only the nprobe closest clusters are searched. Setting nprobe equal to nlist gives exact results. A common rule of thumb is to set nlist near the square root of the number of vectors for small collections.

IVF indexes must be trained on a representative set of vectors. If many new documents are added later, the index should be rebuilt so the clusters still reflect the data.

6. Generation and prompting

The prompt given to the language model should tell it to answer using only the supplied context and to say that it does not know when the context is insufficient. Including the page number with every chunk lets the model, and the reader, see where each statement came from. Retrieving four to six chunks is a typical starting point; more chunks add noise and cost more tokens.

7. Evaluating a RAG system

Retrieval and generation should be evaluated separately. For retrieval, build a small set of questions with known answer pages and measure how often the correct page appears in the top k results, known as recall at k. For generation, check faithfulness, meaning that every claim in the answer is supported by the retrieved text, and relevance, meaning that the answer actually addresses the question.

Common failure modes include chunks that cut a table in half, scanned PDFs with no text layer, questions that need information from several distant pages, and models that ignore the context and answer from memory.